



CodeCrunch

[Home](#) | [My Courses](#) | [Browse Tutorials](#) | [Browse Tasks](#) | [Search](#) | [My Submissions](#) | [Logout](#) | Logged in as: **e0148630**

CS2030 (2420) Practical Assessment #1

Tags & Categories

Tags:

Categories:

Related Tutorials

Task Content

Book of Life

Mr. Get A. Life has come to the realization that he is approaching the middle stage of life. In order to leave a legacy behind, he has begun to write his own autobiography. As it is still very much in progress, he does not know how many chapters nor pages. All he has now are scraps of writings which, many a time, he needs to refer back and forth. Let us help him organize his life story!

Task

Your task is to model a book of possibly infinite chapters and infinite pages. Pages of a book can be flipped via an external viewer.

Take Note!

This task comprises a number of levels. You are required to complete ALL levels.

The following are the constraints imposed on this task. In general, you should keep to the constructs and programming discipline instilled throughout the module.

- Write each class or interface in its own file. Do not use single letter names for classes or interfaces.
- Ensure that ALL object properties and class constants are declared `private final`
- Ensure that ALL classes/interfaces are NOT cyclic dependent.
- ONLY the following java libraries ARE allowed:
 - `java.util.List`
 - `java.util.Optional`
 - `java.util.stream.Stream`
 - functional interfaces from `java.util.function`
- The following are NOT allowed:
 - `var`, `null`, `default`, `enum`
 - `instanceof` (except checking for an instance of its own class)
 - Optional methods: `isPresent`, `isEmpty`, `get` and its variants (`orElse`, `orElseGet`, `orElseThrow`), as well as `equals` and `hashCode`
- There is no need to use bounded wildcards.
- You are NOT allowed to define anonymous inner classes; if necessary, define lambdas instead.
- Other usual restrictions:
 - Use only `&&`, `||` and `!` in logical expressions. DO NOT use bitwise operators.

- You are NOT allowed to use * wildcard imports.
- You are NOT allowed to use method references ::
- You are NOT allowed to define array constructs, e.g. `String[]` or using ellipsis, e.g. `String...`
- You are NOT allowed to use exception handling
- You are NOT allowed to use Java reflection, i.e. `Object::getClasses` and other methods from `java.lang.Class`

In the following level specifications, you will be guided on the non-private methods to define. Any other method you create yourself must be declared `private`.

A minimal [Pair](#) class (not a record, but behaves similarly) has been provided for you. DO NOT replace this with your own version.

Level 1

You are given the following [Printable](#) interface. DO NOT replace this with your own version.

```
interface Printable {
    public void show();
}
```

Write an implementation class `Para` that encapsulates the paragraphs of text, each with an associated paragraph number. The contents of the paragraph can only be output (via `System.out.println`) using the `show()` method.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> new Para(1, "This is a paragraph")
$.. ==> paragraph

jshell> new Para(1, "This is a paragraph").show()
1: This is a paragraph

jshell> Printable p = new Para(1, "This is a paragraph")
p ==> paragraph

jshell> p.show()
1: This is a paragraph
```

Level 2

Write a `Page` class with a constructor that takes in a `String` comprising paragraphs separated by the newline `\n` character. Here is an example of how you can break the lines using the `lines()` method.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> "one\ntwo\nthree".lines()
$.. ==> java.util.stream.ReferencePipeline$Head@..

jshell> "one\ntwo\nthree".lines().forEach(x -> System.out.println(x))
one
two
three
```

Note that the `Page` class should also implement the `Printable` interface.

```
jshell> Printable p = new Page("This is a paragraph.\nThis, is another paragraph.")
p ==> page

jshell> p.show()
1: This is a paragraph.
2: This, is another paragraph.
```

Level 3

Write a class `Chapter` that creates a chapter comprising pages via two constructors, one that takes in a single page, and the other that takes in pages as a `Stream` of pages.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> Page pp1 = new Page("This is a paragraph.\nThis, is another paragraph.")
pp1 ==> page

jshell> Page pp2 = new Page("A page with only one paragraph.")
pp2 ==> page

jshell> new Chapter(pp1)
$.. ==> chapter

jshell> new Chapter(Stream.of(pp1, pp2))
$.. ==> chapter

jshell> new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("page " + x.toString())))
$.. ==> chapter
```

The last example illustrates a chapter comprising an infinite number of pages, each with only one paragraph, specifically page 1, page 2, ...

Pages of a chapter can be flipped back and forth. Flipping the pages is implemented via a `View` that is returned from the `view()` method. The implementation of this method is enforced by implementing the `Viewable` interface.

You are given the following [Viewable](#) interface. DO NOT replace this with your own version.

```
interface Viewable {
    public View view();
}
```

Here is an example of calling the `view` method.

```
jshell> new Chapter(Stream.of(pp1, pp2)).view()
$.. ==> view
```

Note that the purpose of the `View` is to flip pages, regardless of whether they are pages of a chapter or a book, or even a series of books. Include a `pages()` method that returns the pages of the chapter as a stream. This is useful for you to verify that the pages are formed correctly.

```
jshell> new Chapter(Stream.of(pp1, pp2)).view().pages()
$.. ==> java.util.stream.ReferencePipeline$Head@..

jshell> new Chapter(Stream.of(pp1, pp2)).view().pages().forEach(x -> x.show())
1: This is a paragraph.
2: This, is another paragraph.
1: A page with only one paragraph.
```

Moreover, a view is printable, i.e. the `show()` method can be invoked.

```
jshell> new Chapter(Stream.of(pp1, pp2)).view().show()
1: This is a paragraph.
2: This, is another paragraph.
```

You will notice that only the contents of the first page `pp1` is shown. Flipping to the next page is invoked via the `next()` method.

```
jshell> new Chapter(Stream.of(pp1, pp2)).view().next().show()
1: A page with only one paragraph.
```

You can flip back to the previous page using the `prev()` method.

```
jshell> new Chapter(Stream.of(pp1, pp2)).view().next().prev().show()
1: This is a paragraph.
2: This, is another paragraph.
```

Since we may have an infinite number of pages in a chapter, flipping is implemented lazily:

- every `next()` method call advances the page by one with no upper bound;
- a `prev()` method call flips back one page;
- if a `prev()` is called on the first page, it remains at the first page.

Note that by the time the `show()` method is invoked and flipping has gone beyond a finite number of pages, the last page will be shown.

```
jshell> new Chapter(Stream.of(pp1, pp2)).view().next().next().show()
1: A page with only one paragraph.

jshell> new Chapter(Stream.of(pp1, pp2)).view().next().next().prev().show()
1: A page with only one paragraph.

jshell> new Chapter(Stream.of(pp1, pp2)).view().next().next().prev().prev().show()
1: This is a paragraph.
2: This, is another paragraph.
```

Moreover, a chapter of empty pages will simply show blank. Flipping with `next()` and `prev()` will not have any effect.

```
jshell> new Chapter(Stream.of()).view().show()

jshell> new Chapter(Stream.of()).view().next().show()

jshell> new Chapter(Stream.of()).view().prev().show()

jshell>
```

Level 4

Now we are ready to implement the `Book` class. A book may comprise one or more chapters.

As with chapters, define two `Book` constructors: one that takes in a single chapter, and the other that takes in a `Stream` of chapters.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> Page pp1 = new Page("This is a paragraph.\nThis, is another paragraph.")
pp1 ==> page

jshell> Page pp2 = new Page("A page with only one paragraph.")
pp2 ==> page

jshell> new Book(new Chapter(pp1))
$.. ==> book

jshell> new Book(new Chapter(Stream.of(pp1, pp2)))
$.. ==> book

jshell> new Book(Stream.of(
...>     new Chapter(Stream.of(pp1, pp2)),
...>     new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("para " + x)))))
$.. ==> book
```

A book may also have infinite number of chapters.

```
jshell> new Book(Stream.generate(() -> new Chapter(Stream.of(pp1, pp2))))
$.. ==> book
```

And just like a chapter, a book is also viewable.

```
jshell> new Book(Stream.of(
...>     new Chapter(Stream.of(pp1, pp2)),
...>     new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("para " + x)))).
...> view().show()
1: This is a paragraph.
2: This, is another paragraph.
```

In the above example, only the first page of the first chapter in the book is shown. Here is the sample run that demonstrates invoking a series of `next()` methods.

```
jshell> new Book(Stream.of(
...>     new Chapter(Stream.of(pp1, pp2)),
...>     new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("para " + x)))).
...> view().next().show()
1: A page with only one paragraph.

jshell> new Book(Stream.of(
...>     new Chapter(Stream.of(pp1, pp2)),
...>     new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("para " + x)))).
...> view().next().next().show()
1: para 1
```

In the last example, the first page of the second chapter is shown.

As with the previous level, `prev()` works the same way when flipping the pages of a book.

```
jshell> new Book(Stream.of(
...>     new Chapter(Stream.of(pp1, pp2)),
...>     new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("para " + x)))).
...> view().next().next().prev().show()
1: A page with only one paragraph.
```

The astute reader may have noticed that every test requires new books and chapters to be created. This is due to the nature of streams being operated only once.

Furthermore, just like the case for chapters with empty pages, books with empty chapters will also show blank.

```
jshell> new Book(Stream.of()).view().show()

jshell>
```

Level 5

Write an unnamed `Main.java` with the `show` method that takes in a (possibly infinite) stream of chapters and books, followed by two integers `m` and `n` where

- `m` denotes the m^{th} chapter/book in the stream (the first being 1);
- `n` denotes the page number of chapter/book `m`.

The method will output the corresponding page.

Don't forget to include the necessary import statements and the following program fragment:

```
void main() {}
```

Here is a sample run.

```
$ javac --enable-preview --release 21 Main.java
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
```

```

...> 1, 1)
1: first

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 2, 1)
1: second

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 2, 2)
1: third

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 2, 3)
1: fourth

```

Anytime m or n is out of bounds, not found is output.

```

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 0, 1)
not found

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 1, 2)
not found

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 2, 4)
not found

jshell> show(Stream.of(new Chapter(new Page("first")),
...>     new Book(Stream.of(new Chapter(new Page("second")), new Chapter(
...>         Stream.of(new Page("third"), new Page("fourth"))))),
...> 3, 1)
not found

```

Here is a more contrived example of an infinite stream of books passed into the method.

```

jshell> show(Stream.generate(() -> new Book(
...>     Stream.of(new Chapter(Stream.of(pp1, pp2)),
...>         new Chapter(Stream.iterate(1, x -> x + 1).map(x -> new Page("para " + x)))))),
...> 10, 7)
1: para 5

```

Submission (Practice)

Your Files:

BROWSE

SUBMIT (only .java, .c, .cpp, .h, .jsh, and .py extensions allowed)

To submit multiple files, click on the Browse button, then select one or more files. The selected file(s) will be added to the upload queue. You can repeat this step to add more files. Check that you have all the files needed for your submission. Then click on the Submit button to upload your submission.